

Library Management System using Object-Oriented Programming Concepts in Java

Objective / Aim

To understand and implement the core Object-Oriented Programming (OOP) concepts in Java, including Classes, Objects, Constructors, Encapsulation, Inheritance, Polymorphism, Abstraction, and Interfaces by developing a Library Management System.

Algorithm / Logic Description

Part A – Classes and Objects

1. Create a class named **Book**.
2. Declare Book ID, Book Name, Author, and Price.
3. Create an object and display its details.

Part B – Constructors and Encapsulation

1. Declare data members as **private**.
2. Create default and parameterized constructors.
3. Create Getter and Setter methods.
4. Access private data through methods.

Part C – Inheritance

1. Create a parent class **Person**.
2. Create child classes **Student** and **Faculty**.
3. Inherit the properties of Person.
4. Display inherited information.

Part D – Polymorphism

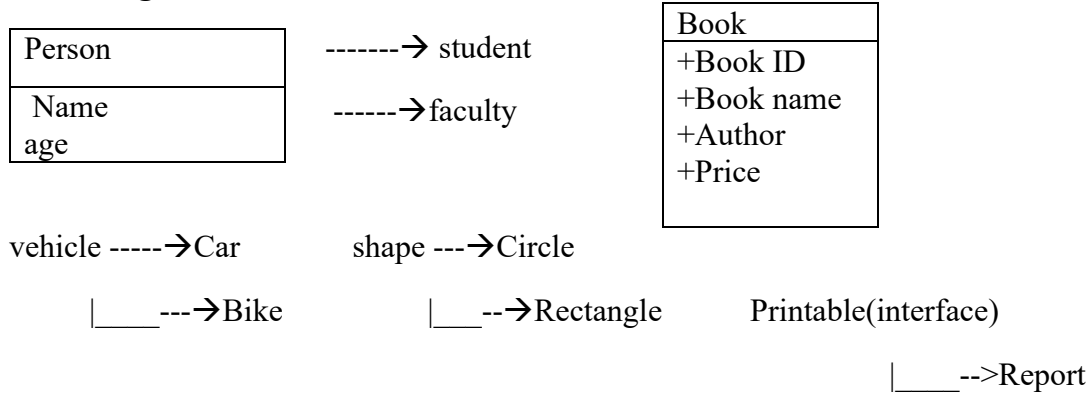
1. Implement method overloading using an **Area** class.
2. Implement method overriding using **Vehicle**, **Car**, and **Bike** classes.
3. Display overridden methods.

Part E – Abstraction and Interfaces

1. Create an abstract class **Shape**.
2. Implement draw() in **Circle** and **Rectangle**.
3. Create an interface **Printable**.

4. Implement the interface in **Report** class.
5. Display all outputs.

UML Diagram :



Source code :

```

package tasks;

public class LibraryManagementSystem {

    public static void main(String[] args) {

        System.out.println("----- BOOK DETAILS -----");

        Book b1 = new Book(101,"Java Programming","James Gosling",550);

        b1.display();

        System.out.println();

        System.out.println("----- STUDENT -----");

        Student s = new Student("Mohana",19,101);

        s.display();

        System.out.println();

        System.out.println("----- FACULTY -----");

        Faculty f = new Faculty("Ramesh",40,"Java");

        f.display();

        System.out.println();
    }
}
  
```

```

        System.out.println("----- METHOD OVERLOADING -----");
        Area a = new Area();
        System.out.println("Circle Area = "+a.calculateArea(5.0));
        System.out.println("Rectangle Area = "+a.calculateArea(5,4));
        System.out.println("Triangle Area = "+a.calculateArea(10.0,8.0));
        System.out.println();
        System.out.println("----- METHOD OVERRIDING -----");
        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();
        v1.display();
        v2.display();
        System.out.println();
        System.out.println("----- ABSTRACTION -----");
        Shape c = new Circle();
        Shape r = new Rectangle();
        c.draw();
        r.draw();
        System.out.println();
        System.out.println("----- INTERFACE -----");
        Report rp = new Report();
        rp.print();
    }
}

class Book{
    private int bookId;
    private String bookName;
    private String author;

```

```
private double price;

// Default Constructor

public Book(){

    bookId = 0;

    bookName = "";

    author = "";

    price = 0;

}

// Parameterized Constructor

public Book(int id, String name, String author, double price){

    this.bookId = id;

    this.bookName = name;

    this.author = author;

    this.price = price;

}

// Getters

public int getBookId(){

    return bookId;

}

public String getBookName(){

    return bookName;

}

public String getAuthor(){

    return author;

}

public double getPrice(){

    return price;

}
```

```

    }
    // Setters
    public void setBookId(int id){
        bookId = id;
    }
    public void setBookName(String name){
        bookName = name;
    }
    public void setAuthor(String a){
        author = a;
    }
    public void setPrice(double p){
        price = p;
    }
    public void display(){
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

class Person{
    String name;
    int age;
    public Person(String name,int age){
        this.name=name;
        this.age=age;
    }
}

```

```

    }

    public void display(){
        System.out.println("Name : "+name);
        System.out.println("Age : "+age);
    }
}

class Student extends Person{
    int rollNo;

    public Student(String name,int age,int rollNo){
        super(name,age);
        this.rollNo=rollNo;
    }

    public void display(){
        super.display();
        System.out.println("Roll No : "+rollNo);
    }
}

class Faculty extends Person{
    String subject;

    public Faculty(String name,int age,String subject){
        super(name,age);
        this.subject=subject;
    }

    public void display(){
        super.display();
        System.out.println("Subject : "+subject);
    }
}

```

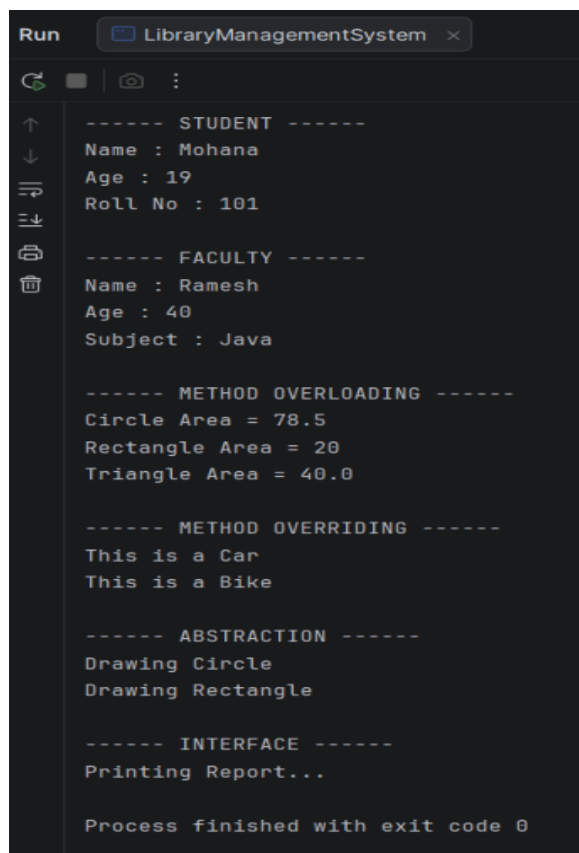
```

    }
}
class Area{
    public double calculateArea(double radius){
        return 3.14*radius*radius;
    }
    public int calculateArea(int length,int breadth){
        return length*breadth;
    }
    public double calculateArea(double base,double height){
        return 0.5*base*height;
    }
}
class Vehicle{
    public void display(){
        System.out.println("This is a Vehicle");
    }
}
class Car extends Vehicle {
    @Override
    public void display() {
        System.out.println("This is a Car");
    }
}
class Bike extends Vehicle{
    @Override
    public void display(){

```

```
        System.out.println("This is a Bike");
    }
}
abstract class Shape{
    abstract void draw();
}
class Circle extends Shape{
    @Override
    void draw(){
        System.out.println("Drawing Circle");
    }
}
class Rectangle extends Shape{
    @Override
    void draw(){
        System.out.println("Drawing Rectangle");
    }
}
interface Printable{
    void print();
}
class Report implements Printable{
    @Override
    public void print(){
        System.out.println("Printing Report...");
    }
}
```


Output :



```
Run  LibraryManagementSystem x
----- STUDENT -----
Name : Mohana
Age : 19
Roll No : 101

----- FACULTY -----
Name : Ramesh
Age : 40
Subject : Java

----- METHOD OVERLOADING -----
Circle Area = 78.5
Rectangle Area = 20
Triangle Area = 40.0

----- METHOD OVERRIDING -----
This is a Car
This is a Bike

----- ABSTRACTION -----
Drawing Circle
Drawing Rectangle

----- INTERFACE -----
Printing Report...

Process finished with exit code 0
```